

PRÁCTICA B1 · BLOQUE I – ADQUISICIÓN

Despliegue del ecosistema AIoT

DURACIÓN	2 sesiones · 4 h	MODALIDAD	Laboratorio · equipos de dos
SESIONES	1 y 2 · 28 de enero y 4 de febrero	HARDWARE	M5StickC Plus 2
APORTA AL PROYECTO	Dispositivo y conectividad	HITO	Prepara H1

01 Objetivos

Al finalizar esta práctica, el alumno habrá adquirido las siguientes competencias técnicas:

- Gestión de Firmware IoT (Edge Computing): Comprender el ciclo de vida del software embebido, desde el "flasheo" de un microcontrolador ESP32 hasta su programación remota mediante UiFlow 2.0 Web IDE.
- Desarrollo Móvil Ágil (RAD): Configurar un entorno de desarrollo rápido (B4A) para desplegar aplicaciones nativas Android, dominando las técnicas de depuración inalámbrica y por bus USB (ADB).
- Entornos Virtuales de Data Science: Configurar un entorno de desarrollo en PyCharm (Licencia UVigo) aislado mediante venv, evitando conflictos de dependencias, y preparando el stack de librerías científicas (pandas, scikit-learn).
- Ciberseguridad en IoT: Desplegar y asegurar un Cluster MQTT Privado en HiveMQ Cloud, gestionando Listas de Control de Acceso (ACLs) y comprendiendo la importancia del cifrado TLS/SSL en el transporte de datos.

02 Competencias y resultados de aprendizaje

Esta práctica desarrolla los siguientes resultados de aprendizaje de la memoria verificada del título:

- **RA2** · Ser capaz de adquirir señales reales utilizando hardware específico. El bloque I es el que lo desarrolla.
- **RA5** · Programar sistemas empotrados para adquirir y preprocesar señales unidimensionales y multidimensionales.
- **RA8** · Dotar a los sistemas empotrados o a las redes de sensores de capacidad de interacción con la nube. Es aquí donde se cubre la parte de MQTT.

En cuanto a competencias, la práctica trabaja la CG2, resolución de problemas con iniciativa y autonomía, y la CE7, necesidades de adquisición, almacenamiento y procesamiento de datos en el contexto de IoT y sus plataformas. El trabajo en equipos de dos desarrolla además la TR2.

03 Material y datos

LISTA DE MATERIALES Y SOFTWARE

Asegúrate de tener todo lo siguiente antes de empezar. La preparación del entorno es el 50% del éxito en ingeniería.

Hardware:

- 1x M5StickC Plus 2: Un potente dispositivo IoT basado en el chip ESP32-PICO-D4. Incluye pantalla LCD, IMU de 6 ejes, micrófono y batería.
- Nota Crítica: Debes usar el cable USB-C suministrado. Muchos cables de cargadores de móvil son "solo carga" y no transmiten datos, lo que impedirá el flasheo.
- 1x Smartphone Android: Se requiere Android 8.0 o superior.
- Requisito: Debe tener el "Modo Desarrollador" activado para permitir la instalación de Apps propias (APK) y el acceso avanzado a los sensores WiFi.
- PC con Windows: Requerido específicamente para el entorno B4A.

Software (Descargas e Instalación):

- M5Burner: Herramienta oficial para grabar el firmware base en la memoria flash del ESP32. Descargar [aquí](#)
- UiFlow2 Web IDE: Entorno de desarrollo en la nube (Cloud IDE). No requiere instalación, corre en el navegador Chrome/Edge gracias a la API WebSerial. Acceder [aquí](#)
- PyCharm Professional: El IDE de Python más potente del mercado. Los alumnos de la UVigo tienen licencia gratuita. Activar Licencia Estudiante
- B4A (Basic4Android): Entorno RAD que simplifica drásticamente el desarrollo Android comparado con Android Studio/Kotlin. Descargar [aquí](#)
- MQTT Explorer: La "navaja suiza" para visualizar tráfico MQTT. Imprescindible para depurar. Descargar [aquí](#)

04 Fundamento teórico

INTRODUCCIÓN Y CONTEXTO TEÓRICO

Bienvenidos a la primera sesión práctica de la asignatura. Antes de sumergirnos en la complejidad matemática de los algoritmos de Inteligencia Artificial (como las Redes Neuronales o los algoritmos de k-Vecinos más Cercanos), es imperativo construir una infraestructura sólida capaz de alimentar esos modelos.

¿Por qué es esto crucial?

La Inteligencia Artificial aplicada al mundo real no vive en el vacío de un servidor; necesita "ojos" y "oídos" físicos para percibir el entorno. En ingeniería de datos, existe el principio "Garbage In, Garbage Out" (Si

entra basura, sale basura). Si nuestra infraestructura de adquisición de datos es inestable, lenta o insegura, el mejor modelo de IA del mundo fallará.

En esta práctica configuraremos la pila completa de AIoT (Artificial Intelligence of Things), una arquitectura distribuida que simula un entorno de producción real:

- El Borde (The Edge - M5StickC): Hardware embebido que digitaliza la realidad física. Aquí aprendemos sobre restricciones de energía y computación.
- La Interfaz (Frontend - Android): La capa de interacción humano-máquina. El smartphone actúa como un sensor avanzado (IMU, WiFi, Barómetro) y como actuador (Voz, Vibración).
- El Cerebro (Backend - Python): Donde reside la lógica pesada. Usaremos Python por ser el estándar de facto en Data Science.
- El Sistema Nervioso (Middleware - MQTT Seguro): La red que transporta los mensajes. Usaremos MQTT sobre TLS, el estándar industrial para comunicaciones M2M (Machine-to-Machine) seguras.

Este enfoque modular nos permitirá, en el futuro, cambiar una pieza (ej. mejorar el sensor) sin romper el resto del sistema.

05 Procedimiento

PARTE A: EL MICROCONTROLADOR (M5STICKC PLUS 2)

El M5StickC viene de fábrica "vacío" o con una demo genérica. Para poder programarlo en Python (MicroPython) y conectarlo a la nube, necesitamos instalarle un sistema operativo específico: UiFlow 2.0.

Paso A.1: Quemado del Firmware (Flashing)

Este proceso sobrescribe la memoria ROM del dispositivo.

- Conecta el M5StickC al PC por USB. Deberías oír el sonido de conexión de Windows.
- Ejecuta M5Burner. Si es la primera vez, puede pedirte instalar drivers (CP210x o CH9102); hazlo si el dispositivo no aparece.
- En el menú lateral izquierdo, selecciona la familia STICKC.
- Usa el buscador para encontrar la imagen UIFLOW_STICKC_PLUS2.
- ¡Cuidado!: Existen versiones para el "StickC" original (gris) y el "Plus" (naranja antiguo). Asegúrate de elegir PLUS2 para evitar errores gráficos en la pantalla.
- Pulsa Download (para bajar la imagen al PC) y luego Burn.
- Configuración WiFi (Crítico): Antes de quemar, aparecerá una ventana pidiendo SSID y Password.
- Introduce las credenciales del laboratorio o de tu móvil (Hotspot).
- ¿Por qué? El dispositivo necesita internet para conectarse a los servidores de M5Stack y recibir el código que escribas en el navegador.
- Al finalizar, el dispositivo se reiniciará. En la pantalla verás el logo de "UiFlow 2.0" y, si ha conectado al WiFi, verás un estado "Connected" o su dirección MAC.

Paso A.2: Vinculación con el Web IDE

UiFlow 2.0 permite programar el hardware sin cables (OTA - Over The Air) o por cable (WebSerial).

- Abre uiflow2.m5stack.com en un navegador moderno (Chrome o Edge).
- (Opcional pero recomendado) Regístrate en M5Stack para guardar tus proyectos en la nube.
- En la esquina inferior izquierda, busca el icono de conexión (un rayo azul o una cadena).
- Selecciona el método de conexión:
- Opción A (WebSerial - Cable USB): Es la más rápida y estable para empezar. Selecciona el puerto COM.
- Opción B (WiFi - Inalámbrica): Si el dispositivo está en la misma red, aparecerá listado por su dirección MAC.
- Selección del Modelo: En el centro de la pantalla, asegúrate de que el dispositivo visualizado es un M5StickC Plus 2. Si no, cámbialo en la configuración del proyecto.

Paso A.3: "Hola Mundo" en MicroPython

Aunque UiFlow ofrece programación visual por bloques (Blockly), en ingeniería usaremos MicroPython para tener control total sobre los hilos y interrupciones.

- En la parte inferior del IDE, cambia de la pestaña "Blockly" a la pestaña "Python" (icono < >).
- Analiza y copia el siguiente código de prueba:

```

import os, sys, io
import M5
from M5 import *
from hardware import *

label0 = None

def setup():
    """Configuración inicial del hardware"""
    global label0
    M5.begin()

    # Limpiar pantalla pintándola de gris oscuro (Hex 0x222222)
    Widgets.fillScreen(0x222222)

    # Crear etiqueta: Texto, X, Y, Capa, Color Texto (Blanco), Color Fondo, Fuente
    label0 = Widgets.Label("AINI WEB IDE", 10, 30, 1.0, 0xffffffff, 0x222222, Widgets.FONTS.DejaVu18)

    # Salida por consola serial (visible en el navegador)
    print("Sistema iniciado desde la Nube. Batería OK.")

def loop():
    """Bucle principal (se repite infinitamente)"""
    M5.update()
    # Aquí irá la lógica de lectura de sensores en el futuro

if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print("Error crítico en tiempo de ejecución")

```

- Dale al botón Run (Play) en la barra superior. El código se compila, se envía al dispositivo y se ejecuta inmediatamente. Deberías ver el texto en la pantalla LCD.

PARTE B: EL MÓVIL (B4A - BASIC4ANDROID)

Android es un sistema operativo complejo basado en Linux. B4A abstrae esa complejidad, permitiéndonos acceder a sensores y crear interfaces gráficas con un lenguaje sencillo (Basic), compilando a código nativo Java/Kotlin.

Paso B.1: Instalación del Entorno

- Instala B4A en tu PC Windows.
- Abre el B4A SDK Manager (se instala junto con B4A). Este gestor descarga las herramientas de Google.
- Instala/Actualiza los paquetes: commandlinetools, android-33 (API Level 33) y build-tools.
- En el IDE, ve a Tools > Configure Paths. Verifica que las rutas a javac.exe y android.jar están en verde.

Paso B.2: Conexión de Depuración (Elegir Estrategia)

Para probar tu App, necesitas conectarla al IDE.

- Opción A (B4A Bridge - Inalámbrica):
- Ventaja: Permite moverte libremente por el laboratorio para probar la navegación.
- Proceso: Instala la App "B4A-Bridge" en el móvil. Conecta móvil y PC a la misma WiFi. Pulsa "Start" en el móvil. En el IDE: Tools > B4A Bridge > Connect (introduce la IP).
- Opción B (Cable USB - ADB):
- Ventaja: Funciona aunque el WiFi esté saturado y permite ver logs del sistema completos.
- Proceso: Activa "Depuración por USB" en las Opciones de Desarrollador del móvil. Conecta el cable. Acepta el aviso de seguridad RSA en la pantalla del móvil. El IDE detectará el dispositivo automáticamente (asegúrate de desconectar el Bridge primero).

Paso B.3: App "Hola Sensores"

Vamos a crear una App mínima para verificar que tenemos control sobre el hardware del teléfono (vibrador).

- File > New > Default.
- Sustituye el código por el siguiente script:

```

Sub Process_Globals
    'Variables que sobreviven si la App pasa a segundo plano
End Sub

Sub Globals
    'Componentes de la interfaz gráfica
    Dim btnTest As Button
    Dim lblInfo As Label
    Dim Phone1 As Phone 'Objeto para acceder a hardware del sistema
End Sub

Sub Activity_Create(FirstTime As Boolean)
    'Constructor de la Interfaz de Usuario

    'Inicializamos y configuramos el botón
    btnTest.Initialize("btnTest")
    btnTest.Text = "PROBAR VIBRACIÓN HÁPTICA"
    'Añadimos a pantalla: X=10%, Y=40%, Ancho=80%, Alto=60 dip (density-independent pixels)
    Activity.AddView(btnTest, 10%x, 40%y, 80%x, 60dip)

    'Inicializamos la etiqueta de estado
    lblInfo.Initialize("")
    lblInfo.Text = "Sistema AINI Listo. Esperando input..."
    lblInfo.Gravity = Gravity.CENTER
    lblInfo.TextSize = 20
    lblInfo.TextColor = Colors.Black
    Activity.AddView(lblInfo, 0, 20%y, 100%x, 50dip)

    Activity.Color = Colors.White
End Sub

Sub btnTest_Click
    'Evento al pulsar el botón
    Dim p As Phone

    'Vibrar durante 500ms (medio segundo)
    p.Vibrate(500)

    'Feedback visual
    lblInfo.Text = "Hardware OK: Motor detectado"
    lblInfo.TextColor = Colors.Green
    ToastMessageShow("Test ejecutado correctamente", False)
End Sub

```

- Pulsa F5 para compilar. La APK se generará, se firmará con una clave de depuración y se instalará en el móvil automáticamente.

PARTE C: EL CEREBRO (PYCHARM + UVIGO)

En la asignatura usaremos PyCharm Professional. Aprovecharemos la licencia educativa de la Universidad de Vigo para tener acceso a las herramientas avanzadas de Data Science y visualización de bases de datos.

Paso C.1: Instalación y Licencia

- Accede a JetBrains Student y solicita tu licencia usando tu correo institucional (@alumnos.uvigo.es).
- Descarga e instala PyCharm Professional.

- Al abrirlo por primera vez, inicia sesión con tu cuenta de JetBrains para activar la licencia.

Paso C.2: Configuración del Proyecto AINI (Backend)

Es vital trabajar con entornos virtuales (venv) para evitar conflictos entre librerías.

- Abre PyCharm y pulsa New Project.
- Nombra el proyecto: AINI_Backend.
- En Interpreter type, selecciona "Project venv" (Entorno virtual). Esto creará una carpeta venv dentro de tu proyecto con una copia aislada de Python.
- Pulsa Create.

Paso C.3: Instalación del Stack de IA

Necesitamos librerías específicas para IoT y Machine Learning.

- Abre la pestaña Terminal en la parte inferior de PyCharm. Asegúrate de que al principio de la línea de comandos pone (venv), indicando que estás en el entorno virtual.
- Ejecuta el siguiente comando:

```
pip install paho-mqtt pandas scikit-learn matplotlib jupyter
```

- paho-mqtt: Cliente para el protocolo MQTT.
- pandas: Manipulación de datos (DataFrames).
- scikit-learn: Algoritmos de Machine Learning (k-NN, Random Forest).
- matplotlib: Visualización de gráficas.
- Verificación: Crea un archivo check_env.py, pega este código y ejecútalo (Click derecho -> Run):

```
import sklearn
import paho.mqtt.client as mqtt
import pandas as pd

print("=== INFORME DE SISTEMA ===")
print(f"Scikit-learn versión: {sklearn.__version__}")
print(f"Pandas versión: {pd.__version__}")
print("Cliente MQTT: Instalado correctamente")
print("=====")
```

PARTE D: LA RED IOT SEGURA (HIVEMQ CLOUD)

En un entorno universitario o profesional, la seguridad no es opcional. No usaremos brokers públicos abiertos. Desplegaremos un Cluster Privado con autenticación y cifrado.

Paso D.1: Crear Cluster en HiveMQ Cloud

- Entra en HiveMQ Cloud y regístrate para el plan "Free Serverless".
- Sigue el asistente para crear un cluster (selecciona proveedor AWS o Azure, es indiferente para nuestro uso).

- El cluster tardará unos minutos en aprovisionarse.

Paso D.2: Gestión de Identidades (ACLs)

El login de la web de HiveMQ es para administrar, NO para conectar dispositivos. Debes crear credenciales para tus sensores.

- Ve a la pestaña Access Management.
- Crea un nuevo usuario:
- Username: alumno_iot (o tu nombre).
- Password: secreto_seguro (¡Anótala!).
- Permissions: Publish & Subscribe.

Paso D.3: Datos de Conexión Segura

En la pestaña Overview, localiza:

- Cluster URL: xxxxxxxx.s1.eu.hivemq.cloud (Esta es tu dirección única en internet).
- Port: 8883 (Este es el puerto para MQTTS - MQTT Seguro sobre TLS). Nota: El puerto 1883 es para texto plano y suele estar desactivado en cloud.

Paso D.4: Verificación con MQTT Explorer

- Abre MQTT Explorer. Crea una nueva conexión.
- Protocol: mqtt:// (TLS Activado).
- Host: Tu URL de HiveMQ. Port: 8883.
- Username / Password: Las que creaste en el paso D.2.
- Pulsa Connect. Si ves el árbol de tópicos (vacío al principio), tu infraestructura en la nube está operativa.

06 Entregables

TAREA FINAL DE INTEGRACIÓN (ENTREGABLE)

Para demostrar que has dominado la pila completa, debes conectar el móvil (Frontend) con la Nube (Middleware) de forma segura.

- Mantén MQTT Explorer abierto y conectado a tu cluster. Suscríbete al topic # (todo).
- En B4A, modifica el Activity_Create para usar la conexión segura SSL.

Código B4A para Conexión Segura:

'IMPORTANTE: Marca la librería "jMQTT" en la pestaña Libraries (derecha del IDE)

```

Sub Globals
    Dim mqtt As MqttClient
    Dim opciones As MqttConnectOptions
End Sub

Sub Activity_Create(FirstTime As Boolean)
    'Configurar opciones de conexión
    opciones.Initialize("alumno_iot", "secreto_seguro") 'Usa tus credenciales del paso D.2
    opciones.CleanSession = True

    'Configurar el Broker
    'PREFIXO ssl:// -> Fuerza el uso de TLS/SSL
    'PUERTO 8883 -> Puerto seguro estándar
    Dim broker As String = "ssl://tu-cluster-id.s1.eu.hivemq.cloud:8883"
    Dim clienteID As String = "Android_" & DateTime.Now 'ID único para evitar desconexiones

    mqtt.Initialize("mqtt", broker, clienteID)

    'Intentar conectar (Es asíncrono, no bloqueará la app)
    mqtt.Connect2(opciones)
End Sub

Sub mqtt_Connected (Success As Boolean)
    If Success Then
        ToastMessageShow(";Conectado a la Nube Segura!", True)
        'Enviar mensaje de prueba
        mqtt.Publish("aini/pruebas", Serializator.ConvertObjectToBytes("Hola desde Android"), 0,
False)
    Else
        ToastMessageShow("Error de conexión. Revisa credenciales.", True)
        Log(LastException)
    End If
End Sub

```

Contenido del PDF Entregable:

Sube un único PDF al Campus Virtual con las siguientes evidencias visuales:

- Foto: Tu M5StickC mostrando tu nombre o "AINI WEB IDE" en la pantalla.
- Captura de Pantalla (Móvil): App B4A ejecutándose, preferiblemente mostrando el mensaje "Hardware OK" o el Toast de "Conectado".
- Captura de Pantalla (PC - PyCharm): Consola mostrando la salida exitosa del script check_env.py con las versiones de las librerías.
- Captura de Pantalla (PC - MQTT Explorer): Interfaz conectada al puerto 8883 de HiveMQ Cloud, mostrando (si lo lograste) el mensaje enviado desde el móvil.

07 Criterios de evaluación

La práctica se evalúa sobre el entregable de integración y sobre lo comprobado en el aula al cierre de la segunda sesión.

Criterio	Peso	Qué se comprueba
Dispositivo operativo	25 %	El M5StickC arranca con UIFlow 2.0, se conecta al WiFi y ejecuta código propio.
Entorno de desarrollo	20 %	PyCharm con entorno virtual aislado y las dependencias instaladas; B4A operativo.
Red MQTT segura	30 %	Cluster propio con TLS, credenciales por equipo y listas de control de acceso configuradas.
Verificación de extremo a extremo	25 %	Un mensaje publicado desde el dispositivo se ve en el cliente de escritorio.

SOBRE LAS CREDENCIALES

Cada equipo usa las suyas y no las escribe dentro del código que entrega. Los esqueletos vienen con marcadores del tipo `___MQTT_USER___` y así deben entregarse.